Claude.ai Workflow for FPGA Architecture Authoring

A reference for adopting LLM-assisted FPGA architecture work, written from the architect's chair on the BPMS SFU project.

The methodology and the framework documents are the load-bearing thing here, not the LLM. The LLM is a force multiplier for an architect who already has a clear document framework, source-of-truth discipline, and review gates. Without that, an LLM accelerates the production of plausible-but-wrong content.

Premise

The architect's job is to produce documents — FAD, Module Spec, ICD, ADR, RTM — that survive review by a senior RTL designer and a verification engineer without requiring follow-up. LLMs are useful here because:

1. **Drafting throughput.** Pulling structured prose from source docs is mechanical; the architect's time is better spent on decisions, tradeoffs, and boundary calls.
2. **Cross-doc consistency.** Every spec change ripples to RTM, ICDs, and the FAD. LLMs catch the ripples reliably at scale.
3. **Adversarial review.** A fresh model with no draft history catches gaps the author can no longer see.

The risk is plausible-but-wrong output. The discipline below exists to address that.

Three-LLM pipeline

```
Draft (Claude.ai) → Review (ChatGPT) → Decide (engineer) → Apply (Claude Code) → Check
```

Role	Tool	Purpose	Does NOT
`fpga_arch` (drafter)	Claude.ai Project	Draft FAD / MS / ICD / ADR sections from source docs and notes	Invent requirements; resolve TBDs without source; sign off
`fpga_arch_reviewer` (reviewer)	ChatGPT custom project	Adversarial review: find gaps, check methodology, challenge assumptions	Rewrite documents; make architecture decisions
`repo_operator` (executor)	Claude Code	Repo-aware edits: apply accepted changes, consistency checks, generate scripts	Change frozen/baselined docs without instruction; silently resolve markers

Why three different tools, not one: the reviewer must be loaded with different context (sign-off criteria, framework conventions) than the drafter (source docs, current FAD), and a fresh model with no draft history is a better adversarial reviewer. The operator runs against the actual repo, which neither chat tool does well.

The engineer is the only signoff authority. LLM "verdicts" are advisory.

Claude Project setup

Each role gets its own Project (Claude or ChatGPT), configured with three layers:

1. Project instructions

A single block defining:

- Role, scope, deliverables (the 5-document framework, authoring order)
- Source-of-truth documents and their precedence (e.g. "SFU-001 primary, ARCH-001 secondary; when they conflict, prefer the more specific and more recent")
- Placeholder markers and citation discipline (see below)
- Output conventions (where files land, frontmatter format, kebab-case)
- Response structure for non-trivial questions: interpretation → facts (cited) → gaps → proposed approach → verification impact → next decisions
- What's explicitly out of scope

Mine is ~250 lines. The point is to encode the *engineering contract* the LLM should follow on every turn — not just answer style. Iterate it over the first 1–2 weeks; it stabilises quickly.

2. Project knowledge (uploaded files)

What I keep loaded:

- **Source-of-truth docs** — system architecture (`ARCH-001`) and SFU architecture (`SFU-001`)
- **Methodology docs** — the five `01-..05-*.md` files (project methodology, architect workflow, RTL workflow, execution flow, sign-off criteria)
- **`arch/README.md`** — framework definition: lifecycle states, review gates, naming, conventions
- **`state.md`** — canonical state (see next section)
- **Reference guides** — best-practice methodology, FPGA architect resource guide, anything that shapes how the LLM thinks about the domain

What I deliberately do NOT upload:

- Old chat logs (they're in checkpoints)
- Drafts under active iteration (in-chat context only)
- Templates that aren't yet stable

3. User preferences

Separate from project instructions — these encode tone, format, and engineering defaults that apply across all my professional work, not just this project. (E.g. "assume strong domain knowledge", "prefer `Q(I.F)` notation", "be direct, prefer engineering realism over elegance".)

Keep these separate from project instructions: they're about *me*, not about the *project*.

---

state.md discipline

The single most useful file in the system.

Purpose: `state.md` is the *map* — current phase, what exists, what's next, what's blocked. It is not the history. History lives in checkpoints and ADRs.

Discipline:

- Mirror it to the Project knowledge so the LLM always has it.
- Update it at the end of every substantive session.
- Keep it shallow — pointers to checkpoints, not duplicated content.
- Encode in project instructions: "if chat or instructions contradict `state.md`, `state.md` wins."

Sections that work:

- Current phase (one paragraph)
- What exists (table of artefacts with location and status)
- Load-bearing technical facts (the small set of decisions that everything else depends on)
- ADR pipeline (proposed / accepted / superseded, with triggers)
- Task status table
- Active flags (open questions and blockers, each with a resolution trigger)
- Next concrete actions (unblocked / blocked-on-X)
- References (paths, document IDs)
- Recent sessions (one line per checkpoint, pointing to the full one)

Sections that don't:

- Long technical explanations (those belong in the documents being authored)
- Reasoning history (belongs in checkpoints)
- Decision rationale (belongs in ADRs)

The discipline is "single page" — when `state.md` exceeds a screen, you're duplicating content that belongs elsewhere.

Anti-laundering discipline

The risk in multi-LLM workflows: one LLM invents a plausible value, another LLM "validates" it because it has no source either, the engineer accepts it because it sounds right. Six months later it's wrong and untraceable.

Countermeasures, encoded in project instructions and enforced by the reviewer:

Placeholder markers (greppable, mandatory):

- `[TBD: <reason>, <owner>]` — value not yet known; who owns the resolution
- `[STUB: <blocking item>]` — section deliberately empty; name the blocker
- `[ASSUMPTION: <text>, <expiry trigger>]` — chosen without source confirmation; name what will confirm or overturn
- `[INFERRED from <source §>]` — derived from a source doc but not literally stated there

A document with many markers is healthier than a polished document with hidden gaps.

Citation discipline:

- Every factual claim from a parent document carries an inline citation, e.g. `(SFU-001 §6.4)`.
- Claims without citation must carry the matching marker (`[INFERRED]` or `[ASSUMPTION]`).
- The reviewer LLM is instructed to flag any uncited claim.

ADR discipline:

- Every non-trivial choice is recorded in an ADR.
- ADRs are immutable once accepted; superseding requires a new ADR linking the predecessor.
- The body cites the forces that justify the decision and lists alternatives with pros/cons.

The discipline is more important than any single tool. Hidden gaps are worse than visible markers.

Custom skills

A skill is a folder with a `SKILL.md` defining when to trigger and what to produce. It encodes a workflow the LLM should run, not just a style preference.

I built one: **`checkpoint-work`**. Triggered by phrases like *"checkpoint this chat"* or *"save checkpoint"*. It distills the conversation into a structured Markdown note with strict frontmatter, scoped to the project, ready to drop into a wiki.

Why it's useful: without it, summarising a long technical chat is freeform and loses fidelity. With it, every checkpoint has the same shape — decisions, open items, a re-seed block to resume in a new chat. The next session, possibly weeks later, picks up cleanly.

Heuristic for when to build a skill: when the same workflow has been done manually three times. Don't over-engineer skills upfront. Other useful candidates I haven't built yet:

- ADR generation from a chat decision
- ICD skeleton from interface description
- Diff-and-reconcile between architect draft and designer template
- RTM regeneration from FAD + MS files

What this enables

- **Continuity across sessions.** Open a new chat, the LLM has `state.md` + source docs + framework loaded. No re-explaining context. The first turn of a new session is productive.
- **Honest gaps.** Markers force me to admit what's unknown rather than glossing over it. Reviewers and downstream consumers can act on visible gaps.
- **Cross-doc consistency at scale.** The LLM catches *"you said X in FAD §4 but the MS for module Y says Z"* — hard to do manually once you have 20+ documents.
- **Reviewable history.** Every decision lands in an ADR, every chat in a checkpoint, every change in a commit. A new engineer can reconstruct the reasoning.
- **Adversarial pressure.** The reviewer LLM, loaded with sign-off criteria but no draft history, finds blind spots faster than re-reading my own work.

What it does NOT do

- **Sign off on anything.** Every gate (G1–G5) is human-decided. LLM-generated review summaries are advisory.
- **Replace domain judgment.** The LLM is good at structuring known engineering content into clean prose. It is not good at, e.g., choosing between two filter-bank implementations on first principles. That's the architect's call.
- **Catch every drift.** Long chats accumulate inconsistency. The discipline (`state.md`, checkpoints, ADRs) is what catches drift, not the LLM itself.
- **Work without source docs.** Without uploaded source-of-truth, the LLM hallucinates. Project knowledge is non-negotiable.
- **Generate the methodology.** I had to author the framework (5-doc set, gates, lifecycle states, marker conventions) myself before the LLM became useful. The LLM helps execute against a framework; it can't pick one for you.

Bootstrapping checklist

For someone setting this up from scratch:

1. **Decide the document framework.** The 5-doc set (FAD, MS, ICD, ADR, RTM) is one option; arc42, NASA GSFC, or your own variant work too. The point is fixing the set before writing any doc. Without this, every chat reinvents structure.
2. **Write the project instructions.** Encode the framework, source-of-truth precedence, marker conventions, output format, response structure. Iterate over the first 1–2 weeks.
3. **Upload source docs to Project knowledge.** System-level + any reference materials. Re-upload when revised.
4. **Author `state.md`.** Even one paragraph + a task table is enough to start. Update at the end of every session; mirror to Project knowledge.
5. **Set up the reviewer Project.** A different LLM (ChatGPT, or a separate Claude project), loaded with framework + sign-off criteria + the document under review. Brief: adversarial, find gaps, do not rewrite.
6. **Adopt markers and citation discipline from day one.** Retrofitting them is much harder than starting clean.
7. **Build the first custom skill only after the same workflow has been done manually three times.** Premature skills lock in the wrong shape.

Honest caveats

- **Tool churn.** Claude Project features and ChatGPT custom-instruction limits change. The discipline outlives any specific tool.
- **Setup cost.** First two weeks feel slow because you're authoring project instructions, structuring knowledge, and shaping `state.md` alongside actual work. ROI lands around week 3.
- **One-architect-shaped.** This works because I'm the only architect on the deliverables. A team would need: a shared style guide, PR-based review of LLM-generated content, possibly a single shared Project rather than per-engineer setups, and a clear rule on who can update `state.md`.

- **Failure modes I've hit.** (a) Letting `state.md` drift — every session afterwards rebuilds context and burns tokens. (b) Skipping citation on what felt like an "obvious" claim — caught by reviewer two weeks later, traced back to nothing. (c) Over-uploading to Project knowledge — every irrelevant file dilutes attention. Keep it lean.

Suggested adaptation for the BPMS system architect role

The BPMS architect's deliverables (`ARCH-001`, `SFU-001`) are a tier above the SFU FPGA work — they are the source of truth that I cite. A reasonable adaptation:

- **Document framework:** the existing `BPMS-1.0-ARCH-001` / device-level spec set is already the framework. Codify *which sections must exist*, *what counts as design-ready at the system level*, and *what triggers a re-baseline*.
- **Source-of-truth precedence:** customer specs → standards (3GPP, CPRI) → datasheets → derived analysis. The reviewer LLM enforces this.
- **`state.md`-equivalent at the system level:** current architecture phase, open OIs (already tracked), the small set of cross-cutting decisions (e.g. interconnect phase, satellite orientation rules), the next system-level decision needed.
- **Three-LLM pipeline:** identical shape. The drafter has the customer spec + standards + reference designs; the reviewer has the architecture framework and the document under review.
- **Skills worth building:** OI table regeneration; cross-document consistency check (e.g. capacity numbers in `ARCH-001 §21` versus `SFU-001 §12`); ADR scaffolding from a chat decision.

The biggest payoff for system-level work is probably the consistency check — when `ARCH-001` ships at v2.4 and `SFU-001` at v1.6, hand-tracking ripples is the slow part.

References (mine, project-specific — included for grounding)

- `bpms-sfu-fpga-design/arch/README.md` — spec-layer framework
- `bpms-sfu-fpga-design/docs/methodology/01..05-*.md` — execution-layer methodology
- `state.md` — canonical project state
- `BPMS-1.0-ARCH-001 v2.4`, `BPMS-1.0-SFU-001 v1.6` — source-of-truth system documents

Authored: May 2026. Drafted by Claude (`fpga_arch` drafter role) at my request, reviewed and edited for accuracy. The recursion is intentional.